Classifying & Generating Clickbait Spoilers for Text: A Study of Extractive Question Answering and Metric Reliability

Aryan Gandhi
University of Waterloo
a27gandhi@uwaterloo.ca
Student ID: 200783566

Abstract

A clickbait headline works by giving small bits of information while holding out the full information so that the reader has to click to find out more about it. *Clickbait spoiling* is the task of exposing that missing information by generating a short text that satisfies the curiosity induced by a clickbait post. In this report I study and discuss the two-part version of this problem from the original competition's Clickbait Spoiling Corpus. The first part is deciding what kind of answer a headline needs (e.g. a short *phrase*, a longer *passage*, or a *multi*-part texts), and then producing the answer itself, where the predicted kind tells the generator how to respond. Our Task 1's classifier reaches a weighted F_1 of **0.759** and our second task generator reaches a METEOR score of **0.447**. Alongside these results, I highlight two findings that I think are more useful than the ranking. The first is that the choice of evaluation metric quietly drives every modelling decision. In this report, I compare the two common generation metrics, BLEU and METEOR, and show that a change which looks bad under one can be exactly the change that helps under the other. The second is that improvements measured on a small validation set often do not carry over to the test set, and I describe simple signals that predict which ones will. I release all of our code and experiment logs, including the approaches that did not work and why.

1 Introduction

Clickbait headlines exploit what psychologists call a "curiosity gap", meaning they reveal just enough to make a reader want the missing details and withhold that detail to compel the user to click on the post (Loewenstein, 1994). *Clickbait spoiling* (Hagen et al., 2022) is the idea of closing that gap automatically by reading the linked article and generating text, so the reader no longer has to click to satisfy their curosity. Following the SemEval-2023 Task 5 formulation (Fröbe et al., 2023), I approach this as two connected sub-tasks over the same data.

Task 1 decides in what form the spoiler should be generated. Some headlines are resolved by a short *phrase* (e.g. a name or a number), some need a full *passage* (a sentence or paragraph of explanation), and some require multiple non-consecutive pieces of text. Because this is a three class classification problem, it is scored with weighted F_1, which is a standard metric for imbalanced classification problems that I explain further in Section 4.2. **Task 2** then produces the spoiler text itself and is scored with METEOR, a metric for comparing generated text to a reference answer. The two tasks are connected: the type predicted in Task 1 tells the Task 2 generator how to format it's answer.

The goal was to build a solid system for both tasks and also to understand why particular choices helped or hurt. This paper makes three main points. It is shown that a simple, well-chosen pipeline (a text classifier for Task 1 and a question-answering model for Task 2) is competitive on both leaderboards. Additionally, I show that the evaluation metric itself is a design decision: optimizing for BLEU and optimizing for METEOR can pull in opposite directions, so picking the right target matters as much as picking the right model. Finally, I document a number of approaches that did not work and explain the reasons. Since these were as instructive as the successes. All code, configurations, and per-experiment logs are publicly available.[1]

[1] https://github.com/aryangandhi/mse641-clickbait-detection

2 Background and Related Work

Clickbait detection, which is the problem of deciding whether a headline is clickbait or not, has a long history (Potthast et al., 2016). Spoiling, actually resolving the headline, was introduced by Hagen et al. (2022), who released the original Clickbait Spoiling Corpus that was used. The researchers framed phrase spoilers as a *question answering* problem and passage spoilers as a *retrieval* problem, and reported two findings that shaped my design. First, question-answering models clearly beat retrieval models at this task. Second, handling each spoiler type with its own strategy (rather than one model for all types) improves results by several points. SemEval-2023 Task 5 (Fröbe et al., 2023) turned this into the two-task competition that I've participated in.

Ideas I borrowed from prior systems. Two SemEval-2023 systems influenced the approach. The winning entry (Sharma et al., 2023) first reduced each article to the handful of paragraphs most relevant to the headline, using a trained ranking model to decide relevance, and used *focal loss* (Lin et al., 2017) (a training objective that pays extra attention to rare classes) to handle the type imbalance. Another strong entry (Pan et al., 2023) reported a useful and slightly surprising result. They found that feeding the classifier only the headline and article title, and dropping the article body, works better than feeding the whole article. They also cleaned and preprocessed the input text before classifying. I adopted this "title-only" input and found that text cleaning is in fact the single largest improvement on Task 1 (Section 4). I also tried a simple version of their relevance step, by keeping only the article paragraphs with the most word overlap with the headline, but this hurt accuracy. This is likely because a plain word-overlap filter is a poor judge of which paragraph actually matters, which is presumably why the winning system used a trained ranker instead.

Generating spoilers. More recent work by Pal et al. (2024) generates spoilers with LongT5 (Guo et al., 2022), a text-generation model that reads an input and *writes* a new answer rather than copying a span from it, and is built to handle long documents. They report very high scores on passage and multi-part spoilers. As Section 4 discusses, I was not able to reproduce those numbers.

Models and metrics I build on. Our models are pre-trained language encoders: BERT (Devlin et al., 2019), RoBERTa (Liu et al., 2019), and DeBERTa (He et al., 2023). These are models trained on large amounts of text that can be adapted ("fine-tuned") to a specific task with comparatively little data. For Task 2 I use them in the style of SQuAD (Rajpurkar et al., 2016), a benchmark where a model is given a question and a passage and must point to the answer inside the passage. For evaluation I use two standard text-generation metrics, BLEU (Papineni et al., 2002) and METEOR (Banerjee and Lavie, 2005), whose differences turn out to matter (see Section 4.6).

3 Approach

The final system for each task is described here and the full progression of experiments is reported in Section 4.

3.1 Task 1: Classifying the Spoiler Type

The classifier reads the headline together with the article title (but not the article body, following Pan et al. (2023)) and predicts one of the three types. I moved from a simple model to a stronger one in stages.

Our baseline is a classical text classifier. It represents each input with TF-IDF features, which describe a text by the words it contains, weighting words by how distinctive they are, together with *character n-grams*, which are short sequences of letters that capture stylistic cues such as punctuation, capitalization, and number formatting. A logistic-regression model then predicts the type (three classes total). This is a sensible baseline because clickbait style is often visible on the surface of the headline.

I then fine-tune a pre-trained encoder (RoBERTa, later DeBERTa) for classification. Because the three types are imbalanced, I weight the training objective so that rarer types count more. Writing $\hat{y}_c$ for the predicted probability of class c and N_c for its frequency, the objective is the class-weighted cross-entropy

$$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{3} w_c \, y_c \log \hat{y}_c, \qquad w_c = \frac{N}{3N_c}. \quad (1)$$

I also tried focal loss, a variant that further contextualizes hard, minority-class examples. As Section 4 shows, this helped on the validation set but

hurt on the test set, likely due to overfitting. Finally, I clean the input text before classifying by removing hyperlinks, replacing hashtags and user mentions with placeholder tokens, stripping emojis, and expanding contractions. The final Task 1 system is a fine-tuned RoBERTa-large encoder trained with the weighted objective of Equation 1 on cleaned, title-only input.

3.2 Task 2: Generating the Spoiler

I treat spoiler generation as an *extractive question-answering* problem. A question-answering model is given a question and a passage of text and predicts the span of the passage that answers the question. I re-purpose this directly: the clickbait headline plays the role of the question, the linked article plays the role of the passage, and the model learns to point at the portion of the article that resolves the headline. This is a natural fit because the answer almost always appears verbatim in the article, in fact, for 99.6% of training examples the ground-truth spoiler is a literal substring of the article.

I fine-tuned roberta-base-squad2, which is a RoBERTa model already trained on the SQuAD question-answering benchmark, on the ground-truth spoiler spans. Starting from a model that already knows how to do extractive QA matters, a plain encoder begins with an untrained answer head and performs far worse. Because some articles can be longer than the model's input limit, I read them with a sliding window and keep the training checkpoint that scores best on the validation set.

Type-conditioned decoding. The type predicted in Task 1 changes how I turn the model's output into a final answer, following the type-aware idea of Hagen et al. (2022). The strategy for each type is:

- *phrase*: return the single best-scoring span, capped at a short length.
- *passage*: return the *entire paragraph* that contains the best span, rather than the span alone.
- *multi*: return the several best non-overlapping spans, joined together.

The passage rule comes from how METEOR scores answers. Because METEOR rewards when the output includes the reference text (recall) more than being concise (precision), returning the whole paragraph, which is likely to contain the reference text, beats returning a short fragment of it, even though the paragraph includes extra words. I found that for passage spoilers the score is almost entirely determined by whether I pick the right paragraph. If a is the fraction of the time I select a paragraph that actually contains the answer, then

$$\text{METEOR}_{\text{passage}} \approx 0.77\,a + 0.11\,(1-a), \quad (2)$$

that is, a correctly chosen paragraph scores about 0.77 and a wrong one about 0.11. Section 4.8 returns to this.

4 Experiments and Results

4.1 Dataset

I used the Clickbait Spoiling Corpus as provided by the competition, with a fixed split of $3{,}200$ training, 400 validation, and 400 test examples (the test labels and spoilers are withheld and scored only through the leaderboard). The types are imbalanced in training set with the class distribution being 42.7% *phrase*, 39.8% *passage*, and 17.5% *multi*. Each example contains the clickbait post, the linked article, and for the training and validation splits, the ground-truth spoiler and its type are included.

4.2 Evaluation Metrics

The two tasks use different metrics because they are different kinds of problems. Task 1 is classification, so I score it with **weighted** F_1. The F_1 score is the harmonic mean of precision and recall for a class, however, the weighted version averages the per-class F_1 scores in proportion to how often each type occurs. For this problem weighted F_1 score should be used rather than plain accuracy because the types are imbalanced and because of this the accuracy can look high while the model quietly fails on the rare classes (e.g. multi), whereas weighted F_1 only looks good if the model handles all three types reasonably well despite this.

Task 2 is text generation, so we need a metric that compares a produced answer to a reference text (i.e. ground truth). I use METEOR, the competition's official metric. Unlike exact match, METEOR gives partial credit, it matches words that share a stem or are synonyms and it weights recall (how much of the reference the answer covers)

#	System	Val F_1	Test F_1
001	TF-IDF + logistic regression	0.527	0.540
003	+ headline character n-grams	0.626	–
007	+ RoBERTa (blended)	0.695	0.706
012	DeBERTa-v3-base	0.700	0.729
013	RoBERTa-large + focal loss	0.716	0.703
014	RoBERTa-large + weighted loss	0.742	0.732
015	**+ text cleaning**	**0.749**	**0.759**

Table 1: Task 1 progression (weighted F_1). Our best test score is 0.759 and a baseline of 0.540.

above precision (how concise the answer is while matching the reference). This suits spoiling where the point is to convey the withheld information, not to reproduce the exact wording. I also compute BLEU, an older overlap-based metric that focuses on precision and use it as a point of comparison. Section 4.6 explains why the comparison is informative.

4.3 Experimental Setup

All experiments use a fixed random seed and the same evaluation harness, so that every run is logged with its configuration and score. Transformers are fine-tuned with the AdamW optimizer, a linearly decaying learning rate, and gradient clipping. The Task 1 classifiers use an input length of 256 tokens, a learning rate of $1e{-}5$, and up to five or six epochs. The Task 2 model uses a learning rate of $3e{-}5$ and three epochs. For every model I keep the checkpoint that scores best on the validation set. Training was done on a personal laptop, which limited the largest model I could use to RoBERTa-large. Section 5 reflects on this.

4.4 Task 1 Results

Table 1 traces the progression. The largest early gain comes from adding character n-grams to the baseline ($+0.078$), which confirms that a lot of the signal is stylistic and lives on the surface of the headline. Moving to a fine-tuned transformer gives far better results and our best result of **0.759** on the test set comes from RoBERTa-large trained with the weighted objective on cleaned input.

Two details are worth noting. Blending the linear model with the transformer helped RoBERTa slightly but *hurt* the stronger DeBERTa model, because by that point the transformer already captured the stylistic cues the linear model was contributing. And the *multi* class was the hardest throughout (its per-class F_1 is 0.68, against 0.77 for the other two types), since it is the rarest and is easily confused with the passage type.

#	System	Val	Test
002	Question answering, no fine-tuning	–	0.214
005	Fine-tuned QA + type decoding	–	0.440
008	+ full-paragraph passages	0.426	0.445
010	**+ tuned per-type decoding**	**0.471**	**0.447**

Table 2: Task 2 progression (METEOR). Our best test score is 0.447 with a baseline of 0.214.

4.5 Task 2 Results

Table 2 traces Task 2. The two biggest gains are fine-tuning the question-answering model on the ground-truth spoiler spans and adding type-conditioned decoding. Returning the full paragraph for passage spoilers and tuning the per-type decoding reached a test METEOR score of **0.447**.

4.6 The Metric Matters: BLEU versus METEOR

The competition scores METEOR, but BLEU is another popular and familiar NLP metric, I tracked both for comparison which turned out to be very revealing. When I changed the passage decoder to return the whole paragraph (experiment 008), METEOR *rose* by 0.018 while BLEU *fell* from 28.5 to 22.2. The reason is exactly the trade-off between the two metrics, since returning the whole paragraph adds words, which BLEU penalizes as imprecise, but which METEOR rewards for covering more of the reference. Had I been optimizing for BLEU, I would have rejected the change that most improved the metric the competition was actually using for grading. The lesson is that the metric is not a neutral scorer, it silently decides which changes count as progress and is opinionated in what matters. Additionally, an argument can be made that precision should be more equally weighted to recall than METEOR currently allows, to encourage more concise spoilers.

4.7 Validation Gains Do Not Always Transfer

A recurring theme was that an improvement on the 400-example validation set did not reliably carry over to the test set, and occasionally reversed (Table 3).

Two patterns explain most of this. First, a balanced set of predictions was a better sign of real improvement than the validation score itself. Focal loss raised validation F_1 by predicting the rare *multi* type more aggressively, but on the test

Change	ΔVal	ΔTest	Outcome
Task 2 decoding sweep	$+0.044$	$+0.003$	barely transferred
Task 1 focal loss	$+0.015$	-0.027	reversed
Task 1 weighted loss	$+0.041$	$+0.003$	small
Task 1 text cleaning	$+0.007$	$+\mathbf{0.027}$	test beat val

Table 3: Validation improvements transferred inconsistently to the test set.

set those extra guesses were mostly wrong and the score fell; the plainer weighted-loss model, whose predictions were more balanced, transferred cleanly. Second, changes that made the input cleaner or the method more robust, rather than fitting the small validation set, transferred best, since text cleaning was the only change whose test gain actually exceeded its validation gain.

4.8 Error Analysis

Because passages are where most of our lost points sit, I looked at them closely on the 154 passage-type validation examples, and I found a lot about where the system stands and what would move it.

The passage bottleneck: picking the right paragraph. Two facts stand out. The ground-truth spoiler is present somewhere in the article every single time, so nothing is actually missing from the input. Yet the paragraph our system returns contains that spoiler only 54.5% of the time. To put it plainly, the answer is always there and we simply reach for the wrong paragraph almost half the time. This is also where Equation 2 comes from. To explain it further, I measured the average METEOR separately for the two cases and found that when the returned paragraph does contain the spoiler the score averages 0.77, and when it does not it averages only 0.11. The overall passage score is just the mixture of those two outcomes weighted by how often each happens, which is exactly Equation 2. At our 54.5% selection rate it predicts about 0.47, which matches the 0.46 we actually observe.

Two separate gaps to a perfect score. That decomposition shows the distance to a perfect 1.0 is really two different gaps. The large one is *selection*, which involves raising the fraction of correctly chosen paragraphs from 0.55 toward 1.0, this is worth the most by far. The smaller one is *granularity*, even when we pick the right paragraph, we return the *whole* paragraph, which is much longer than the spoiler itself, only about a third of the paragraph's length on average. METEOR rewards us for covering the reference but punishes us for the surrounding sentences we drag along, which is why a correctly chosen paragraph averages 0.77 rather than 1.0. So even flawless selection would cap this strategy near 0.77. The true ceiling is slightly lower, about 0.74, because in roughly 4.5% of cases no single paragraph contains the entire spoiler at all. This is because the answer sometimes can span a paragraph break or spills over from the article title, and a one-paragraph answer can never capture all of it. Trimming the paragraph down to just the spoiler sentence would recover some precision but is a tough task. It also risks cutting part of the answer and since METEOR punishes lost coverage more heavily than extra words, over-trimming tends to score worse than leaving the paragraph whole. That reward system is why we return the full paragraph.

Why the selection goes wrong. The reason we miss the right paragraph so often traces back to how the model was trained. A question-answering model learns to find short, precise answer spans, which is ideal for phrase spoilers but is the wrong instinct for passages. Because it is asked to locate a whole passage, it anchors on the single most answer-like sentence and when a plausible but incorrect sentence appears earlier in the text, it commits to that paragraph instead. The four selection methods in Section 4.9 were all attempts to override this instinct with a separate relevance signal, and since none of them beat simply trusting the model's own span, the more promising fix looks like a stronger extractive model rather than a ranker one.

How classifier errors flow into Task 2. Because the Task 1 prediction determines the Task 2 decoding rule, a misclassified type applies the wrong strategy to an input text. To measure how much this costs, I re-ran Task 2 using the true types instead of our 69.5%-accurate predictions, and validation METEOR rose only from 0.471 to 0.510. That 0.039 is the most a perfect classifier could ever add and a realistic classifier would recover only part of it, so good type prediction does add value but only slightly. The errors that do occur are often concentrated on the multi type, which is both the rarest and the easiest to confuse with a passage, which is aligned with it's status as the

hardest class in Task 1.

Two failure modes the metric exposes. Some of our lowest scores say more about the metric than the model. Phrase spoilers show this most clearly because the answer is often a number or a name and if our wording differs at all from the reference we can score zero even when the meaning is right. Multi spoilers show a different story, where the real difficulty is deciding *how many* pieces to return and *which* ones. For example, recovering two of three correct snippets earns partial credit, but the model has no reliable signal for the true count and tends to guess.

A few concrete examples. These quirks are easier to see in real outputs. For a *phrase* spoiler, the post "You won't believe what Samir Nasri has done…" has the ground-truth spoiler *dying his hair sky blue*, which the model recovers exactly. A revealing phrase failure shows the metric strictness above: for a post about how much to tip, the ground truth is *20%* but the model answers *between $5 and $15*, which is the right idea yet shares no words with the reference and so scores zero. A *passage* example shows the selection problem directly: the ground-truth spoiler is *some of the plot elements are so disturbing that they are making him feel sick*, but the model returns a different, shorter piece of the article (*too dark*) and misses it. Finally, for a *multi* spoiler whose answer has three parts (*Alan Rickman & Rupert Grint CBGB*), the model recovers two of them, which is a partial success.

4.9 What Did Not Work

Several reasonable-looking ideas failed and I think the reasons are as useful as the successes (Table 4). The clearest pattern concerns paragraph selection for passages. I tried four different ways to pick the right paragraph (word-overlap similarity, aggregating the model's span scores per paragraph, a combination of the two, and a separately trained ranking model), and all four did worse than simply trusting the question-answering model's own best span. This independently reproduces Hagen et al. (2022)'s finding that question answering beats retrieval for this task. I also tried generating spoilers with a text-generation model instead of extracting them, but a smaller model reached only 0.31 METEOR on passage and multi spoilers, against 0.42 for extraction.

What I tried	Why it did not help
Keeping only word-overlap paragraphs (Task 1)	word overlap is a poor judge of relevance
Focal loss (Task 1)	over-predicted the rare class; helped val, hurt test
Generating instead of extracting (Task 2)	tended to paraphrase; extraction scored higher
Four paragraph-selection methods (Task 2)	all lost to the QA model's own span
Returning several paragraphs (Task 2)	extra coverage was cancelled by extra noise

Table 4: Approaches that did not improve results, and the reason in each case.

5 Conclusion and Future Work

I built a two-task clickbait-spoiling system, a text classifier for the spoiler type and a fine-tuned question-answering model for the spoiler text, that performed well across both tasks in the competition. I came away with two lessons that generalize beyond this competition. The evaluation metric is arguably the most important design choice, since optimizing for BLEU and optimizing for METEOR reward different behaviour, so choosing the right target is as consequential as choosing the right model. And a small validation set is an unreliable guide, since improvements that come from cleaner inputs or more balanced predictions transfer to the test set far more dependably than improvements squeezed out by tuning to the validation numbers. However, this result could be different if the validation set was very large.

The clearest path forward on Task 2 is better paragraph selection for passage spoilers, since perfect selection would raise our score from about 0.46 to 0.74. However, keep in the mind every selection method I tried lost to the extractive model, so a stronger extractive model is the more promising direction than a ranker. A larger classifier such as DeBERTa-v3-large, combined with our text cleaning, is a natural next step for Task 1. Additionally, in future work I would have trained these larger models on more powerful cloud GPUs from the start. I did not realize early enough how limiting and slower training is on a personal laptop than on the cloud infrastructure used in prior work.

References

Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186.

Maik Fröbe, Tim Gollub, Benno Stein, Matthias Hagen, and Martin Potthast. 2023. SemEval-2023 task 5: Clickbait spoiling. In *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, pages 2275–2286.

Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontanon, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. 2022. LongT5: Efficient text-to-text transformer for long sequences. *Findings of NAACL*.

Matthias Hagen, Maik Fröbe, Artur Jurk, and Martin Potthast. 2022. Clickbait spoiling via question answering and passage retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7025–7036.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations (ICLR)*.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.

George Loewenstein. 1994. The psychology of curiosity: A review and reinterpretation. *Psychological Bulletin*, 116(1):75–98.

Sayantan Pal, Souvik Das, and Rohini K. Srihari. 2024. Mitigating clickbait: An approach to spoiler generation using multitask learning. *arXiv preprint arXiv:2405.04292*.

Ronghao Pan, José Antonio García-Díaz, Francisco García-Sánchez, and Rafael Valencia-García. 2023. Chick adams at SemEval-2023 task 5: Using RoBERTa and DeBERTa to extract post and document-based features for clickbait spoiling. In *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, pages 624–628.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL*, pages 311–318.

Martin Potthast, Sebastian Köpsel, Benno Stein, and Matthias Hagen. 2016. Clickbait detection. In *Advances in Information Retrieval (ECIR)*, pages 810–817.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of EMNLP*, pages 2383–2392.

Anubhav Sharma, Sagar Joshi, Tushar Abhishek, Radhika Mamidi, and Vasudeva Varma. 2023. Billybatson at SemEval-2023 task 5: An information condensation based system for clickbait spoiling. In *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, pages 1878–1889.